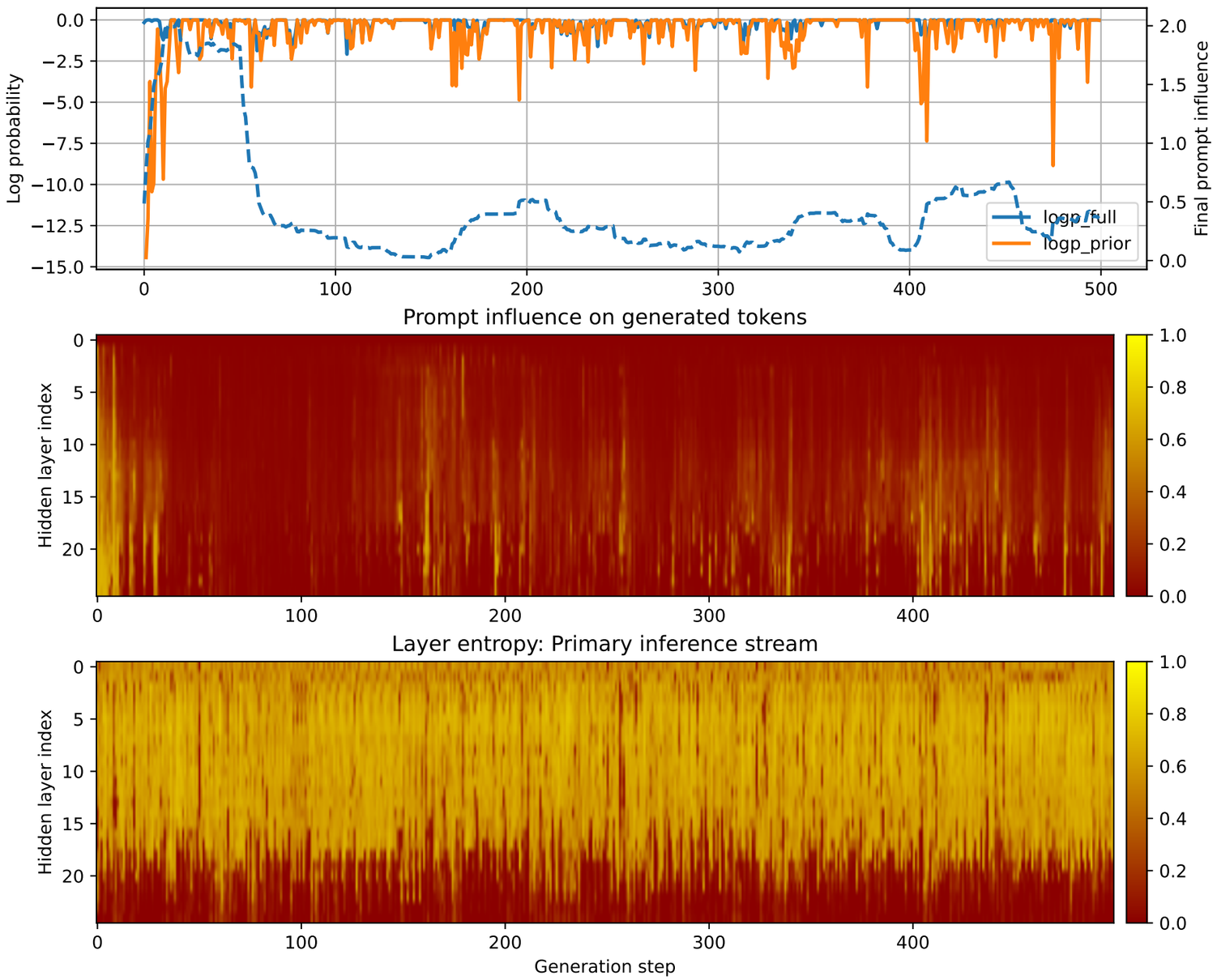
Prompt 1




```
PROMPT:
Task ID: PYC090  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC090  Your code must contain this required symbol:
def compute_confusion_matrix  Implement this
task: write function
compute_confusion_matrix(y_true,y_pred,labels)..  Start your
answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
# CHECK_TOKEN: compute_confusion_matrix  # Python implementation
for compute_confusion_matrix function  def
compute_confusion_matrix(y_true, y_pred, labels):     """
Compute the confusion matrix for a given set of
true labels (y_true) and predicted labels (y_pred)      using the
confusion matrix formula:
confusion_matrix(y_true, y_pred) =      | true_positives |
true_negatives | false_positives | false_negatives
|
|--------------------|--------------------|--------------------|--
    | y_true_i
| y_pred_i           | y_true_i          | y_pred_i           |
    | y_true_j          | y_pred_j
| y_true_j          | y_pred_j           |     | ...
   | ...               | ...
| ...                |      Parameters:     y_true (list): List
of true labels (one per row)     y_pred
(list): List of predicted labels (one per row)     labels (list):
List of corresponding labels (one per row)
Returns:     confusion_matrix (list): A list of confusion
matrices for each label, where each element is a 2x2
matrix representing the number of true positives, true negatives,
false positives, and false negatives for
each label.     """     # Initialize confusion matrix
confusion_matrix = [[0 for _ in range(4)] for _ in
range(len(labels))]      # Compute confusion matrix for each
label     for i in range(len(labels)):
for j in range(len(labels)):             # Get true positive,
true negative, false positive, and false
negative counts             true_positives = sum([1 for true in
y_true if true == label[i]])
true_negatives = sum([1 for true in y_true if true != label[i]])
          false_positives = sum([1 for true
in y_pred if true == label[i]])             false_negatives =
sum([1 for true in y_pred if true != label[i]])
# Add confusion matrix element             confusion_matrix[i][j]
= (true_positives, true_negatives,
false_positives, false_negatives)      return confusion_matrix  #
Example usage y_true = [1, 0, 1, 1] y_pred =
[1, 1, 0, 1] labels = ['A', 'B', 'C', 'D']  confusion_matrix =
compute_confusion_matrix(y_true,
```